

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO SEMINAR

Kiểm thử cơ sở dữ liệu cho EShop

Giảng viên	Hồ Tuấn Thanh
Sinh viên	23127081 - Nguyễn Phan Hùng Linh
	23127172 - Nguyễn Chí Đức
	23127487 - Thôi Đặng Trường Thọ

Thành phố Hồ Chí Minh, tháng 7 năm 2026

Mục lục

1. Giới thiệu	4
2. Tổng quan công cụ	5
DbUnit	5
Database Rider	5
Tonic.ai	5
3. Kiểm thử NoSQL database	6
3.1 Kiểm thử database SQL	6
3.2 Kiểm thử database NoSQL	6
3.3 Khác biệt chính	7
3.4 Khả năng áp dụng của công cụ	7
3.5 Giải thích thuật ngữ ngắn	7
4. Cài đặt và thiết lập	8
5. Thực hành và các bước demo	9
5.1 Demo DbUnit	9
5.2 Demo Database Rider	9
5.3 Demo Tonic.ai	10
6. Kịch bản kiểm thử EShop	11
Kịch bản A: Seed và kiểm tra user	11
Kịch bản B: Checkout và kiểm tra đơn hàng	11
Kịch bản C: Che giấu dữ liệu user nhạy cảm	11
7. So sánh	13
8. Framework kiểm thử database có thể tái sử dụng	14
8.1 Cấu trúc của framework	14
8.2 Cách 1: dùng framework để seed và kiểm tra database	14
8.3 Cách 2: tester thao tác trên giao diện, framework kiểm tra database	15
8.4 Kết quả mong đợi	17
8.5 Quy trình dùng framework cho website khác có database	17
9. Ưu điểm và hạn chế	19
DbUnit	19
Database Rider	19
Tonic.ai	19

10. Phạm vi kiểm thử ngoài ba công cụ	20
10.1 Kiểm tra dữ liệu và quy tắc nghiệp vụ	20
10.2 Bốn nhóm trường hợp kiểm thử	20
10.3 Kiểm tra câu lệnh SQL và thủ tục lưu trữ	20
10.4 Kiểm tra giao dịch và hoàn tác dữ liệu	21
10.5 Kiểm tra nhiều người truy cập cùng lúc	21
10.6 Áp dụng AI trong database testing	21
11. Các vấn đề đã gặp	23
12. Bài học rút ra	24
13. Kết luận	25
14. Bằng chứng và phụ lục	26
Bằng chứng DbUnit	26
Bằng chứng Database Rider	28
Bằng chứng Tonic.ai	28
15. Tài liệu tham khảo	34

Ngày: 2026-07-19

1. Giới thiệu

Kiểm thử cơ sở dữ liệu rất quan trọng vì nhiều lỗi ứng dụng không chỉ xuất hiện trên giao diện người dùng. Lỗi có thể xảy ra khi dữ liệu được lưu, cập nhật, xóa hoặc kiểm tra trong cơ sở dữ liệu.

Trong seminar này, hệ thống cần kiểm thử (System Under Test - SUT) là dự án EShop tại <https://github.com/ttbhanh/eshop-sut>

EShop là một SUT phù hợp vì đây là ứng dụng mua sắm thông thường. Hệ thống có người dùng, sản phẩm, danh mục, mã giảm giá, giỏ hàng, đơn hàng và chức năng quản trị. Tất cả chức năng này đều phụ thuộc vào hành vi chính xác của cơ sở dữ liệu.

Ba công cụ được nghiên cứu trong báo cáo gồm:

- DbUnit
- Database Rider
- Tonic.ai

Mục tiêu chính là tìm hiểu cách các công cụ này hỗ trợ chuẩn bị dữ liệu kiểm thử, kiểm tra kết quả trong cơ sở dữ liệu và tạo dữ liệu test an toàn hơn.

2. Tổng quan công cụ

Công cụ	Mục đích chính	Được dùng cho
DbUnit	Thiết lập cơ sở dữ liệu bằng file dataset và kiểm tra dữ liệu	Unit test và integration test cho cơ sở dữ liệu
Database Rider	Giúp viết test theo phong cách DbUnit dễ hơn	Bảo trì database test dễ hơn
Tonic.ai	Tạo dữ liệu test an toàn và che giấu dữ liệu nhạy cảm	Quản lý dữ liệu kiểm thử

DbUnit

DbUnit là thư viện kiểm thử dành cho Java. Công cụ có thể nạp một dataset vào cơ sở dữ liệu trước khi test chạy. Sau khi test thực thi, chúng ta có thể kiểm tra cơ sở dữ liệu có chứa dữ liệu mong đợi hay không.

Trong demo, DbUnit sử dụng dataset XML.

Database Rider

Database Rider được xây dựng trên DbUnit. Công cụ cung cấp cách viết database test gọn hơn. Database Rider có thể sử dụng dataset YAML, thường dễ đọc hơn XML.

Trong demo, Database Rider sử dụng dataset YAML.

Tonic.ai

Tonic.ai khác với DbUnit và Database Rider. Đây không phải là công cụ assertion chính. Tonic.ai được dùng để tạo hoặc che giấu dữ liệu phục vụ kiểm thử.

Đối với EShop, chức năng này hữu ích vì bảng **users** có thể chứa dữ liệu nhạy cảm như tên, email, mật khẩu, số điện thoại và địa chỉ.

3. Kiểm thử NoSQL database

Kiểm thử database SQL và NoSQL đều có cùng mục tiêu: chuẩn bị dữ liệu test, chạy hành động của ứng dụng, kiểm tra dữ liệu được lưu đúng, rồi dọn dữ liệu test. Điểm khác nhau là cách dữ liệu được tổ chức.

SQL thường dùng bảng, hàng và cột. Ví dụ bảng **users** lưu người dùng, bảng **orders** lưu đơn hàng. NoSQL có nhiều kiểu hơn, ví dụ document database như MongoDB lưu dữ liệu dưới dạng document gần giống JSON.

3.1 Kiểm thử database SQL

Với SQL database, tester thường kiểm tra:

- 1 Bảng và cột có đúng kiểu dữ liệu không.
- 2 Khóa chính và khóa ngoại có bảo vệ quan hệ dữ liệu không.
- 3 Thao tác thêm, sửa, xóa và truy vấn có đúng không.
- 4 **JOIN** có lấy đúng dữ liệu từ nhiều bảng không.
- 5 Transaction có lưu hoặc hủy dữ liệu đúng không.
- 6 Index có giúp truy vấn nhanh hơn không.

Ví dụ trong EShop, **orders.user_id** phải trỏ tới một user thật trong bảng **users**. Nếu đơn hàng trỏ tới user không tồn tại, dữ liệu bị sai.

3.2 Kiểm thử database NoSQL

Với NoSQL database, các bước kiểm thử thường là:

- 1 Xác định loại NoSQL đang dùng, ví dụ document, key-value hoặc graph.
- 2 Chuẩn bị dữ liệu test bằng document hoặc key mẫu.
- 3 Kiểm tra các field bắt buộc có tồn tại không.
- 4 Kiểm tra kiểu dữ liệu, ví dụ **price** phải là số và **email** phải là chuỗi.
- 5 Kiểm tra dữ liệu lồng nhau, ví dụ một đơn hàng có danh sách sản phẩm bên trong.
- 6 Kiểm tra quan hệ do ứng dụng quản lý, vì nhiều NoSQL database không có foreign key.
- 7 Kiểm tra query và aggregation có trả đúng kết quả không.
- 8 Kiểm tra update có làm mất field cũ hoặc ghi sai document không.
- 9 Kiểm tra consistency sau khi ghi dữ liệu.
- 10 Kiểm tra dữ liệu sau khi đổi schema, vì document cũ và document mới có thể

cùng tồn tại.

Ví dụ với MongoDB, một test có thể tạo document user, tạo document order, chạy checkout, rồi kiểm tra collection **orders** có order mới với đúng **userId**, **totalAmount** và **status**.

3.3 Khác biệt chính

Nội dung	SQL database	NoSQL database
Cách lưu dữ liệu	Bảng, hàng, cột	Document, key-value, graph hoặc wide-column
Schema	Thường cố định và rõ ràng	Linh hoạt hơn, có thể khác nhau giữa các document
Quan hệ dữ liệu	Thường dùng foreign key và JOIN	Thường dùng dữ liệu lồng nhau hoặc reference
Transaction	Phổ biến và mạnh	Có nhưng tùy loại database và phạm vi thao tác
Kiểm tra chính	Row, constraint, JOIN , transaction	Document shape, field, query, consistency
Rủi ro thường gặp	Sai quan hệ bảng, sai constraint	Thiếu field, sai kiểu dữ liệu, document cũ và mới khác nhau

3.4 Khả năng áp dụng của công cụ

Công cụ	Dùng với SQL database	Dùng với NoSQL database
DbUnit	Phù hợp, vì làm việc tốt với JDBC và dữ liệu dạng bảng	Không phù hợp trực tiếp, vì không hiểu document hoặc key-value
Database Rider	Phù hợp, vì được xây trên DbUnit và dùng dataset YAML	Không phù hợp trực tiếp, vì vẫn phụ thuộc DbUnit và JDBC
Tonic.ai	Phù hợp để tạo hoặc che dữ liệu test cho nhiều SQL database	Có thể dùng với một số NoSQL connector như MongoDB và DynamoDB

Vì vậy, DbUnit và Database Rider phù hợp nhất cho database SQL. Nếu kiểm thử NoSQL, nhóm thường cần driver chính thức của database, test container hoặc một database test riêng. Tonic.ai có thể giúp chuẩn bị dữ liệu test an toàn, nhưng nó không thay thế test framework dùng để assert kết quả.

3.5 Giải thích thuật ngữ ngắn

- **Schema:** cấu trúc dữ liệu, ví dụ bảng có cột nào hoặc document có field nào.
- **Document:** một bản ghi kiểu JSON, thường dùng trong MongoDB.
- **Collection:** nhóm nhiều document, gần giống bảng trong SQL.
- **Key-value:** kiểu lưu dữ liệu bằng một key và một value.
- **Foreign key:** khóa dùng để nối dữ liệu giữa hai bảng.
- **JOIN:** câu lệnh SQL dùng để lấy dữ liệu từ nhiều bảng cùng lúc.
- **Transaction:** nhóm thao tác phải thành công hết hoặc hủy hết.
- **Consistency:** mức độ dữ liệu đọc ra có mới và đúng ngay sau khi ghi không.
- **Aggregation:** cách gom, lọc và tính toán dữ liệu trong NoSQL query.
- **Assertion:** điều kiện mà test kiểm tra, ví dụ số lượng order phải bằng 1.

4. Cài đặt và thiết lập

Thiết lập cơ bản được sử dụng cho seminar gồm:

- 1 Cài Node.js cho backend và frontend của EShop.
- 2 Cài Java và Maven cho demo DbUnit và Database Rider.
- 3 Cài công cụ SQLite để kiểm tra cơ sở dữ liệu EShop.
- 4 Clone hoặc mở dự án EShop.
- 5 Chạy **npm install** trong các phần Node.js của dự án.
- 6 Chạy **node database.js** để tạo và seed cơ sở dữ liệu SQLite.
- 7 Tạo project Maven demo DbUnit.
- 8 Tạo project Maven demo Database Rider.
- 9 Chuẩn bị workspace Tonic.ai hoặc quy trình sinh dữ liệu Tonic.ai.

Cơ sở dữ liệu backend của EShop nằm tại:

```
eshop-sut/backend/database.sqlite
```

Backend EShop sử dụng SQLite. File seed chính là:

```
eshop-sut/backend/database.js
```

Cơ sở dữ liệu gồm các bảng như:

- **users**
- **categories**
- **products**
- **orders**
- **order_items**
- **coupons**

5. Thực hành và các bước demo

5.1 Demo DbUnit

Đối với DbUnit, nhóm tạo một project Maven nhỏ.

Các file chính:

```
dbunit-demo/pom.xml
dbunit-demo/src/test/resources/datasets/initial-dataset.xml
dbunit-demo/src/test/java/com/eshop/dbunit/EshopDbUnitTest.java
```

Dataset XML chứa dữ liệu mẫu cho các bảng như **users**, **categories** và **products**.

Test sử dụng DbUnit để:

- 1 Đọc dataset XML.
- 2 Khởi tạo lại cơ sở dữ liệu bằng **CLEAN_INSERT**.
- 3 Query cơ sở dữ liệu bằng JDBC.
- 4 Assert dữ liệu mong đợi tồn tại.

Lệnh chạy:

```
mvn test
```

Kết quả ghi nhận:

```
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Kết quả cho thấy demo DbUnit đã chạy thành công.

5.2 Demo Database Rider

Đối với Database Rider, nhóm tạo một project Maven khác.

Các file chính:

```
database-rider-demo/pom.xml
database-rider-demo/src/test/resources/datasets/eshop-users.yml
database-rider-demo/src/test/java/com/eshop/databaserider/EshopDatabaseRiderTest.java
```

Dataset YAML chứa dữ liệu mẫu cho **users**, **categories** và **products**.

Test sử dụng Database Rider để:

- 1 Đọc dataset YAML.
- 2 Nạp dataset vào SQLite.
- 3 Query bảng **users**.
- 4 Assert bảng có đúng số lượng row mong đợi.

Lệnh chạy:

```
mvn test
```

Kết quả ghi nhận:

```
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Kết quả cho thấy demo Database Rider đã chạy thành công.

5.3 Demo Tonic.ai

Quy trình Tonic.ai Structural được thực hiện như sau:

- 1 Khởi tạo lại cơ sở dữ liệu SQLite của EShop.
- 2 Export các bảng **users**, **products**, **coupons** và **orders** sang CSV.
- 3 Upload các file CSV lên Tonic.ai.
- 4 Tạo file group.
- 5 Cấu hình generator cho các cột nhạy cảm.
- 6 Chạy data generation.
- 7 Tải các file CSV đã generate.
- 8 So sánh dữ liệu gốc và dữ liệu đã generate.

Các cột nhạy cảm quan trọng trong bảng **users**:

Cột	Lý do nhạy cảm
name	Tên người dùng thật
email	Thông tin định danh cá nhân
password	Bí mật đăng nhập
shipping_address	Địa chỉ cá nhân
phone	Số điện thoại cá nhân

Các cột như **id**, **role** và foreign key thường nên được giữ ổn định vì ứng dụng cần chúng để bảo toàn quan hệ giữa các bảng.

Trong demo seminar, Tonic.ai được dùng cho dữ liệu **users**. Kiểm tra chính là so sánh CSV gốc với CSV được generate và xác nhận các giá trị nhạy cảm đã thay đổi.

6. Kịch bản kiểm thử EShop

Demo seminar tập trung vào các kịch bản database testing đơn giản, phù hợp với ba công cụ.

Kịch bản A: Seed và kiểm tra user

Đối với DbUnit và Database Rider, kịch bản là:

Nạp dữ liệu test biết trước vào cơ sở dữ liệu SQLite của EShop và kiểm tra bảng users.

Kịch bản này hữu ích vì đăng nhập, checkout và profile đều cần user hợp lệ. Nếu bảng **users** sai, nhiều chức năng EShop không thể hoạt động.

Kết quả mong đợi:

- 1 Cơ sở dữ liệu được đưa về trạng thái biết trước.
- 2 Dataset tạo hai user: một admin và một user test thông thường.
- 3 Query xác nhận bảng **users** có dữ liệu mong đợi.

DbUnit thực hiện bằng dataset XML:

```
src/test/resources/datasets/initial-dataset.xml
```

Database Rider thực hiện cùng ý tưởng bằng dataset YAML:

```
src/test/resources/datasets/eshop-users.yml
```

Kịch bản B: Checkout và kiểm tra đơn hàng

Kịch bản demo ứng dụng EShop:

Đăng nhập bằng user test, thêm sản phẩm vào giỏ hàng, checkout và kiểm tra bảng orders.

Kết quả mong đợi sau checkout:

- 1 Bảng **orders** có đơn hàng mới.
- 2 **user_id** thuộc về user đang đăng nhập.
- 3 **total_amount** khớp với tổng tiền giỏ hàng.
- 4 **status** mặc định của đơn hàng là **pending**.

Có thể kiểm tra cơ sở dữ liệu bằng SQL:

```
SELECT COUNT(*) FROM orders;  
SELECT user_id, total_amount, status FROM orders;
```

Kịch bản C: Che giấu dữ liệu user nhạy cảm

Đối với Tonic.ai, kịch bản là:

Export users từ SQLite sang CSV, upload CSV lên Tonic.ai, che giấu cột nhạy cảm và tải CSV đã generate.

Kết quả mong đợi:

- 1 **name**, **email**, **password**, **shipping_address** và **phone** được thay đổi.
- 2 Giá trị được generate vẫn giống dữ liệu test thực tế.

- 3 Các cột quan hệ quan trọng như **id** được giữ ổn định.
- 4 Dữ liệu mới có thể dùng an toàn hơn cho demo hoặc staging.

7. So sánh

Tiêu chí	DbUnit	Database Rider	Tonic.ai
Loại công cụ	Thư viện kiểm thử	Wrapper của thư viện kiểm thử	Nền tảng dữ liệu test
Input phổ biến	Dataset XML	Dataset YAML	CSV hoặc nguồn database
Output chính	Kết quả test	Kết quả test	Dữ liệu được generate
Mức độ dễ dùng	Trung bình	Dễ hơn DbUnit	Dễ sau khi làm quen UI
Có thể assert kết quả database	Có	Có	Không, nếu chỉ dùng riêng Tonic
Có thể che giấu dữ liệu nhạy cảm	Không	Không	Có
Cách dùng tốt nhất trong EShop	Reset và kiểm tra trạng thái database	Database test gọn hơn	Dữ liệu test an toàn hơn

DbUnit và Database Rider được dùng để kiểm tra cơ sở dữ liệu có đúng hay không. Tonic.ai được dùng để tạo dữ liệu an toàn hơn. Vì vậy, các công cụ bổ trợ lẫn nhau nhưng không thay thế nhau.

8. Framework kiểm thử database có thể tái sử dụng

8.1 Cấu trúc của framework

Framework dùng Java, Maven, JUnit, JDBC, DbUnit và Database Rider.

Giải thích nhanh:

- **JUnit**: thư viện để chạy test trong Java.
- **JDBC**: cách Java kết nối tới database bằng driver.
- **DbUnit**: thư viện dùng dataset để nạp và kiểm tra dữ liệu database.
- **Database Rider**: thư viện xây trên DbUnit, giúp chạy DbUnit dataset dễ hơn.
- **dataset**: file mô tả dữ liệu test, ví dụ bảng **user** cần có những dòng nào.
- **YAML**: định dạng file dễ đọc, dùng thụt dòng để mô tả dữ liệu.
- **assertion**: điều kiện kiểm tra. Nếu điều kiện sai thì test fail.
- **CRUD**: Create, Read, Update, Delete, nghĩa là tạo, đọc, sửa, xóa dữ liệu.

```
framework/  
|-- pom.xml  
|-- examples/  
|   |-- microblog-database-test.properties  
|   |-- microblog-seed.yml  
|   |-- microblog-ui-expected.yml  
|   |-- microblog-ui-result-check.properties  
|-- src/test/java/org/database/testing/framework/  
|   |-- ConfiguredDatabaseStateTest.java  
|   |-- DatabaseTestConfig.java  
|   |-- JdbcDatabaseTestSupport.java
```

Vai trò của từng phần:

- **DatabaseTestConfig**: đọc file cấu hình **.properties**.
- **JdbcDatabaseTestSupport**: mở JDBC connection, chạy Database Rider, và dùng DbUnit dataset bên dưới.
- **ConfiguredDatabaseStateTest**: test runner chung.
- File **.properties** trong **examples/**: cấu hình database, file seed, file expected, và các cột cần bỏ qua.
- **microblog-seed.yml**: dataset dùng để nạp dữ liệu trực tiếp vào database microblog.
- **microblog-ui-expected.yml**: dataset mô tả kết quả mong đợi sau khi tester thao tác trên giao diện.

8.2 Cách 1: dùng framework để seed và kiểm tra database

Cách này dùng khi tester muốn chuẩn bị dữ liệu nhanh mà không cần nhập bằng giao diện.

Bước 1: lấy project microblog nếu máy chưa có:

```
cd /Users/nguyenphanthangthong/hacking/ctf/learning/Eshop-database-testing  
git clone https://github.com/miguelgrinberg/microblog.git
```

Bước 2: chuẩn bị database microblog:

```
cd /Users/nguyenphanthangthong/hacking/ctf/learning/Eshop-database-testing/microblog  
python3 -m venv .venv
```

```
. .venv/bin/activate
pip install -r requirements.txt
rm -f app.db app.db-wal app.db-shm
.venv/bin/flask --app microblog.py db upgrade
```

Bước 3: chạy framework:

```
cd /Users/nguyenphanthangthong/hacking/ctf/learning/Eshop-database-testing/framework
mvn test -Ddatabase.test.config=examples/microblog-database-test.properties
```

Trong cách này:

- Framework kết nối tới **../microblog/app.db**.
- Database Rider chạy **cleanBefore=true** để dọn dữ liệu cũ trong các bảng liên quan.
- Database Rider dùng DbUnit để nạp **examples/microblog-seed.yml** vào database.
- Database Rider dùng DbUnit để so sánh database thật với **examples/microblog-seed.yml**.

Dataset **microblog-seed.yml** có dữ liệu cho 3 bảng:

- **user**: có **alice** và **bob**.
- **post**: có 1 post của **alice**.
- **followers**: có quan hệ **alice** follow **bob**.

Nếu database sau khi seed khác dataset này, test sẽ fail.

8.3 Cách 2: tester thao tác trên giao diện, framework kiểm tra database

Cách này dùng khi tester muốn kiểm tra dữ liệu sinh ra từ hành động thật trên web.

Bước 1: lấy project microblog nếu máy chưa có:

```
cd /Users/nguyenphanthangthong/hacking/ctf/learning/Eshop-database-testing
git clone https://github.com/miguelgrinberg/microblog.git
```

Bước 2: chuẩn bị database test:

```
cd /Users/nguyenphanthangthong/hacking/ctf/learning/Eshop-database-testing/microblog
python3 -m venv .venv
. .venv/bin/activate
pip install -r requirements.txt
rm -f app.db app.db-wal app.db-shm
.venv/bin/flask --app microblog.py db upgrade
```

Bước 3: chạy web microblog:

```
.venv/bin/flask --app microblog.py run --host 127.0.0.1 --port 5000
```

Mở trình duyệt tại:

<http://127.0.0.1:5000>

Bước 4: thao tác CRUD trên giao diện:

- 1 Create user **alice**:
 - Vào **http://127.0.0.1:5000/auth/register**.
 - Nhập **Username**: **alice**.
 - Nhập **Email**: **alice@example.com**.
 - Nhập **Password**: **password**.

- Nhập **Repeat Password**: password.
 - Bấm **Register**.
- 1 Create user **bob**:
- Vào lại **http://127.0.0.1:5000/auth/register**.
 - Nhập **Username**: bob.
 - Nhập **Email**: bob@example.com.
 - Nhập **Password**: password.
 - Nhập **Repeat Password**: password.
 - Bấm **Register**.
- 1 Login bằng user **alice**:
- Vào **http://127.0.0.1:5000/auth/login**.
 - Nhập **Username**: alice.
 - Nhập **Password**: password.
 - Bấm **Sign In**.
- 1 Create post:
- Ở trang Home, nhập: **Hello from microblog UI database test**.
 - Bấm **Submit**.
- 1 Read dữ liệu:
- Vào **Explore** để thấy post vừa tạo.
 - Vào **Profile** để thấy thông tin user **alice**.
- 1 Update profile:
- Vào **Profile**.
 - Bấm **Edit your profile**.
 - Giữ **Username** là **alice**.
 - Nhập **About me**: I am Alice, updated from web UI.
 - Bấm **Submit**.
- 1 Create quan hệ follow:
- Vào **http://127.0.0.1:5000/user/bob**.
 - Bấm **Follow**.
- 1 Delete quan hệ follow:
- Vẫn ở trang user **bob**, bấm **Unfollow**.
 - Microblog không có nút xóa user hoặc post trên giao diện, nên delete được minh họa bằng việc xóa quan hệ follow.

Bước 5: chạy framework để kiểm tra database:

```
cd /Users/nguyenphanthangthong/hacking/ctf/learning/Eshop-database-testing/framework
mvn test -Ddatabase.test.config=examples/microblog-ui-result-check.properties
```


File **microblog-ui-result-check.properties** không seed và không cleanup. Nó chỉ kiểm tra database hiện tại sau khi tester thao tác trên web.

Framework dùng **examples/microblog-ui-expected.yml** để kiểm tra:

- Bảng **user** có đúng **alice** và **bob**.
- **alice** có **about_me** đúng sau bước update.
- Bảng **post** có đúng post vừa tạo.
- Bảng **followers** rỗng sau bước unfollow.

8.4 Kết quả mong đợi

Nếu database đúng, Maven sẽ báo:

```
Running org.database.testing.framework.ConfiguredDatabaseStateTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Nếu dữ liệu sai, ví dụ tester nhập sai nội dung post, Database Rider/DbUnit sẽ báo bảng nào và cột nào khác expected dataset.

Với project khác, phần Java của framework giữ nguyên. Tester chỉ cần đổi:

- JDBC driver và chuỗi kết nối database.
- Dataset seed nếu muốn nạp dữ liệu trước.
- Dataset expected nếu muốn kiểm tra dữ liệu sau thao tác.
- Danh sách cột cần ignore nếu database có dữ liệu tự sinh.

8.5 Quy trình dùng framework cho website khác có database

Khi gặp một project website mới có database, tester làm theo các bước sau:

- 1 Xác định database thật mà website đang dùng khi chạy test. Ví dụ: SQLite file, PostgreSQL database, MySQL database.
- 2 Xác định JDBC driver cần dùng. **JDBC driver** là thư viện giúp Java nói chuyện với database. Framework đã có sẵn SQLite. Nếu project dùng PostgreSQL hoặc MySQL thì thêm driver tương ứng vào **framework/pom.xml**.
- 3 Tạo file cấu hình riêng cho project trong **framework/examples/**, ví dụ **my-project-ui-check.properties**.

```
db.driver=org.sqlite.JDBC
db.url=jdbc:sqlite:./my-project/app.db
db.user=
db.password=
db.schema=
db.seed=
db.assert.dataset=examples/my-project-ui-expected.yml
db.assert.compare=EQUALS
db.assert.ignoreColumns=id,created_at,updated_at
```

- 1 Tạo dataset seed nếu cần nạp dữ liệu trước khi tester thao tác. Nếu không cần seed thì để **db.seed=** rỗng.
- 2 Tạo dataset expected để mô tả dữ liệu mong đợi sau khi tester thao tác trên web. Dataset này nên kiểm tra các cột quan trọng của nghiệp vụ.

- 3 Bỏ qua các cột động bằng **db.assert.ignoreColumns**. **Cột động** là cột do hệ thống tự sinh, ví dụ **id**, **created_at**, **updated_at**, **token**, **password_hash**.
- 4 Tester chạy ứng dụng web bằng lệnh của chính project đó. Framework không chạy app thay tester.

```
cd /path/to/project
# cài dependency theo hướng dẫn của project
# chạy migration hoặc tạo database theo hướng dẫn của project
# start web app theo hướng dẫn của project
```

- 1 Nếu cần seed database trước khi thao tác UI/API, chạy framework với config seed:

```
cd /Users/nguyenphanthangthong/hacking/ctf/learning/Eshop-database-testing/framework
mvn test -Ddatabase.test.config=examples/my-project-seed.properties
```

- 1 Tester thực hiện CRUD trên giao diện hoặc API của website. **CRUD** nghĩa là tạo, đọc, sửa, xóa dữ liệu.
- 2 Sau khi thao tác xong, chạy framework để kiểm tra database thật:

```
cd /Users/nguyenphanthangthong/hacking/ctf/learning/Eshop-database-testing/framework
mvn test -Ddatabase.test.config=examples/my-project-ui-check.properties
```

- 1 Đọc kết quả Maven. Nếu **BUILD SUCCESS** thì database đúng với expected dataset. Nếu test fail, Database Rider/DbUnit sẽ báo bảng hoặc cột nào khác dữ liệu mong đợi.

9. Ưu điểm và hạn chế

DbUnit

Ưu điểm:

- Kiểm soát mạnh trạng thái cơ sở dữ liệu.
- Phù hợp để học nền tảng database testing.
- Có thể reset dữ liệu trước mỗi test.

Hạn chế:

- Dataset XML có thể dài.
- Mã thiết lập ở mức thấp hơn.
- Người dùng cần hiểu Java, JDBC và bảng trong database.
- Không hỗ trợ native cho NoSQL document hoặc key-value.

Database Rider

Ưu điểm:

- Dataset YAML dễ đọc hơn.
- Giảm một phần mã thiết lập so với DbUnit thuần.
- Phù hợp cho database test cần bảo trì lâu dài.

Hạn chế:

- Vẫn yêu cầu Java và Maven.
- Vẫn cần hiểu khái niệm DbUnit.
- Không thay thế việc hiểu database schema.
- Không hỗ trợ NoSQL native vì vẫn dùng DbUnit và JDBC.

Tonic.ai

Ưu điểm:

- Có thể che giấu dữ liệu nhạy cảm.
- Có thể tạo dữ liệu test trông thực tế.
- Hữu ích khi không nên dùng trực tiếp dữ liệu production.
- Hỗ trợ cơ sở dữ liệu quan hệ và một số connector NoSQL.

Hạn chế:

- Không phải công cụ assertion.
- Cần cấu hình generator chính xác.
- Nếu ID hoặc foreign key thay đổi sai, quan hệ có thể bị phá vỡ.
- Nếu email hoặc password bị che giấu, tài khoản demo mặc định có thể không đăng nhập được.
- Mức hỗ trợ và tính năng NoSQL phụ thuộc connector và license.

10. Phạm vi kiểm thử ngoài ba công cụ

DbUnit, Database Rider và Tonic.ai hỗ trợ chuẩn bị dữ liệu, chạy kiểm tra và tạo dữ liệu test. Tuy nhiên, công cụ không tự quyết định toàn bộ điều kiện đúng hoặc sai của hệ thống. Người kiểm thử vẫn phải đọc yêu cầu, hiểu cấu trúc database và thiết kế các trường hợp kiểm thử phù hợp.

Các nội dung trong mục này là phạm vi kiểm thử bổ sung được đề xuất cho dự án. Chúng không đồng nghĩa với việc nhóm đã thực thi toàn bộ các trường hợp nếu chưa có kết quả chạy hoặc bằng chứng tương ứng.

10.1 Kiểm tra dữ liệu và quy tắc nghiệp vụ

Các giá trị được lưu phải đúng với dữ liệu người dùng đã nhập và đúng với yêu cầu nghiệp vụ. Với EShop, một đơn hàng không chỉ cần được tạo mà còn phải thuộc đúng người dùng, có tổng tiền đúng và có trạng thái ban đầu phù hợp.

Những nội dung cần kiểm tra gồm:

- Giá trị nhỏ nhất, lớn nhất và giá trị vượt giới hạn.
- Trường bắt buộc, giá trị **NULL**, chuỗi rỗng và khoảng trắng.
- Unicode, ký tự đặc biệt và kiểu dữ liệu.
- Khóa chính, khóa ngoại và dữ liệu không được trùng.
- Liên kết dữ liệu giữa các bảng sau khi import hoặc chuyển dữ liệu.
- Các quy tắc tự động như trigger nếu database có sử dụng.

10.2 Bốn nhóm trường hợp kiểm thử

Nhóm	Nội dung cần kiểm tra	Ví dụ
Dữ liệu đầu vào	Giới hạn, để trống, Unicode và kiểu dữ liệu	Tên quá dài, số âm, ngày không hợp lệ
Ràng buộc và liên kết	Khóa chính, khóa ngoại, dữ liệu trùng và mapping	Order tham chiếu tới user không tồn tại
Tốc độ xử lý	Tìm kiếm, nối bảng, phân trang và xử lý nhiều dữ liệu	Query bị chậm hoặc hết thời gian chờ
Cách thực hiện	Giao diện, API, SQL và test tự động	Thao tác checkout rồi kiểm tra bảng orders

Một test chỉ có ý nghĩa khi dữ liệu đầu vào, kết quả mong đợi và môi trường chạy đã được xác định rõ.

10.3 Kiểm tra câu lệnh SQL và thủ tục lưu trữ

Người kiểm thử có thể dựa vào hiểu biết về SQL để tạo các trường hợp làm câu lệnh thất bại hoặc trả về dữ liệu sai. Không chỉ kiểm tra câu SQL có chạy được hay không, cần kiểm tra cả dữ liệu được lưu sau khi câu lệnh chạy.

Các bước đề xuất:

- 1 Chuẩn bị dữ liệu hợp lệ và dữ liệu có chủ đích gây lỗi.

- 2 Chạy câu SQL hoặc thủ tục lưu trữ bằng công cụ quản lý database.
- 3 Kiểm tra cách SQL tương tác với API hoặc thành phần của ứng dụng.
- 4 So sánh dữ liệu thực tế với kết quả mong đợi.
- 5 Xác nhận khi thao tác thất bại thì dữ liệu sai không bị lưu lại.

10.4 Kiểm tra giao dịch và hoàn tác dữ liệu

Một giao dịch có thể gồm nhiều thao tác thêm hoặc cập nhật dữ liệu. Nếu một thao tác thất bại, các thay đổi trước đó của cùng giao dịch phải được hủy để database không rơi vào trạng thái chỉ cập nhật một phần.

Các trường hợp cần kiểm tra:

- Toàn bộ các bước đều thành công và dữ liệu được lưu đầy đủ.
- Bước đầu thành công nhưng bước sau thất bại.
- Mất kết nối hoặc xuất hiện lỗi khi giao dịch đang chạy.
- Sau lỗi, dữ liệu được hoàn tác về trạng thái trước giao dịch.
- Gửi lại cùng một yêu cầu không tạo dữ liệu trùng ngoài mong đợi.

10.5 Kiểm tra nhiều người truy cập cùng lúc

Database có thể xử lý nhiều giao dịch trong cùng một thời điểm. Vì vậy cần kiểm tra tình huống nhiều người hoặc nhiều tiến trình cùng đọc và cập nhật một dữ liệu.

Những rủi ro chính gồm:

- Hai giao dịch cùng cập nhật một bản ghi và làm mất thay đổi của nhau.
- Một giao dịch đọc dữ liệu khi giao dịch khác chưa hoàn thành.
- Dữ liệu bị khóa quá lâu làm request khác phải chờ hoặc hết thời gian.
- Hai giao dịch giữ khóa mà giao dịch kia đang cần, tạo ra deadlock.

Khi xảy ra xung đột, hệ thống cần có cách xử lý rõ ràng như chờ có giới hạn, thử lại, báo lỗi hoặc hoàn tác giao dịch. Kết quả cuối cùng không được làm dữ liệu mâu thuẫn hoặc cập nhật dở dang.

10.6 Áp dụng AI trong database testing

AI có thể hỗ trợ người kiểm thử ở ba giai đoạn:

Giai đoạn	AI có thể hỗ trợ
Chuẩn bị	Gợi ý trường hợp kiểm thử, dữ liệu mẫu và câu SQL tạo dữ liệu
Phân tích	So sánh kết quả, đọc log lỗi và tìm dữ liệu hoặc query bất thường
Viết bản nháp	Soạn test mẫu và danh sách cần kiểm tra để tester xem lại

AI chỉ là công cụ hỗ trợ. Người kiểm thử vẫn phải xác nhận câu SQL, dữ liệu và kết quả mong đợi bằng yêu cầu hệ thống và kết quả chạy thực tế.

Các nguyên tắc an toàn:

- Không gửi dữ liệu production nhạy cảm cho AI.
- Không chạy trực tiếp câu SQL do AI tạo trên database production.
- Chạy thử trên database dành riêng cho kiểm thử hoặc bản sao có thể phục hồi.
- Xem lại quan hệ khóa ngoại và quy tắc nghiệp vụ trước khi dùng dữ liệu AI tạo.
- Không xem câu trả lời của AI là bằng chứng test nếu chưa chạy xác nhận.

11. Các vấn đề đã gặp

Vấn đề	Giải thích ngắn
Thiếu module sqlite3	Dependency Node.js chưa được cài đặt
Lỗi parse Maven pom.xml	Nội dung không hợp lệ bị copy thêm vào cuối pom.xml
DbUnit order test thất bại	Order mong đợi chưa tồn tại
Tonic.ai báo nhiều schema	Các CSV có schema khác nhau bị upload vào cùng một file group
orders.csv rỗng	Chưa có order sau khi reset seed data
Email/password bị mask ảnh hưởng đăng nhập	Credential được generate không còn khớp với luồng demo
Lỗi import/package Database Rider	Tên import Java phải khớp với phiên bản Database Rider

12. Bài học rút ra

Database testing cần dữ liệu ổn định. Nếu cơ sở dữ liệu thay đổi ngẫu nhiên, test sẽ khó đáng tin cậy.

DbUnit và Database Rider hữu ích khi cần chuẩn bị trạng thái database biết trước trước khi test. Hai công cụ cũng hữu ích khi cần kiểm tra database sau một hành động.

Tonic.ai hữu ích khi cần dữ liệu test nhưng không nên dùng dữ liệu nhạy cảm thật. Công cụ bảo vệ quyền riêng tư bằng cách thay thế tên, email, địa chỉ và số điện thoại.

Quan hệ foreign key rất quan trọng. Ví dụ, nếu **orders.user_id** tham chiếu đến **users.id**, cần cẩn thận khi generate hoặc thay đổi ID.

Database test có thể xác minh điều mà screenshot UI không chứng minh được. Ví dụ, sau checkout, UI có thể báo thành công nhưng database test mới xác nhận row thật sự đã được thêm vào bảng **orders**.

Relational và NoSQL có cùng test lifecycle nhưng failure model khác nhau. NoSQL đòi hỏi kiểm tra thêm consistency, partition, replication, document version và quy tắc toàn vẹn ở tầng ứng dụng.

Chạy công cụ thành công chưa đủ để kết luận database an toàn. Cần kiểm tra thêm giới hạn dữ liệu, ràng buộc, giao dịch, hoàn tác và truy cập đồng thời.

AI có thể giúp chuẩn bị test case và phân tích lỗi nhanh hơn, nhưng người kiểm thử vẫn phải xác nhận kết quả bằng yêu cầu và bằng chứng chạy thực tế.

13. Kết luận

DbUnit hữu ích để hiểu nền tảng database testing. Công cụ cho thấy cách nạp dataset và kiểm tra kết quả trong cơ sở dữ liệu.

Database Rider dễ bảo trì hơn vì dataset YAML gọn và workflow test ngắn hơn.

Tonic.ai hữu ích để tạo dữ liệu test an toàn. Công cụ giúp generate hoặc mask dữ liệu để quá trình kiểm thử không làm lộ thông tin nhạy cảm.

Đối với EShop, ba công cụ phối hợp tốt theo các vai trò khác nhau:

- DbUnit và Database Rider giúp xác minh hành vi cơ sở dữ liệu.
- Tonic.ai giúp chuẩn bị dữ liệu test an toàn hơn.
- Kịch bản checkout EShop cho thấy lý do cần kiểm tra database sau hành động của user.
- Các kiểm tra về SQL, rollback, truy cập đồng thời và hiệu năng giúp mở rộng phạm vi ngoài test dataset cơ bản.
- AI có thể hỗ trợ chuẩn bị và phân tích, nhưng không thay thế người kiểm thử hoặc bằng chứng thực tế.

Seminar cũng làm rõ một ranh giới quan trọng: DbUnit và Database Rider là công cụ cho cơ sở dữ liệu quan hệ, trong khi Tonic Structural hỗ trợ cơ sở dữ liệu quan hệ và một số connector NoSQL. Giải pháp có tính di động nên tái sử dụng test harness và workflow, đồng thời giữ schema, dataset và business assertion riêng cho từng project.

14. Bảng chứng và phụ lục

Tài liệu hỗ trợ:

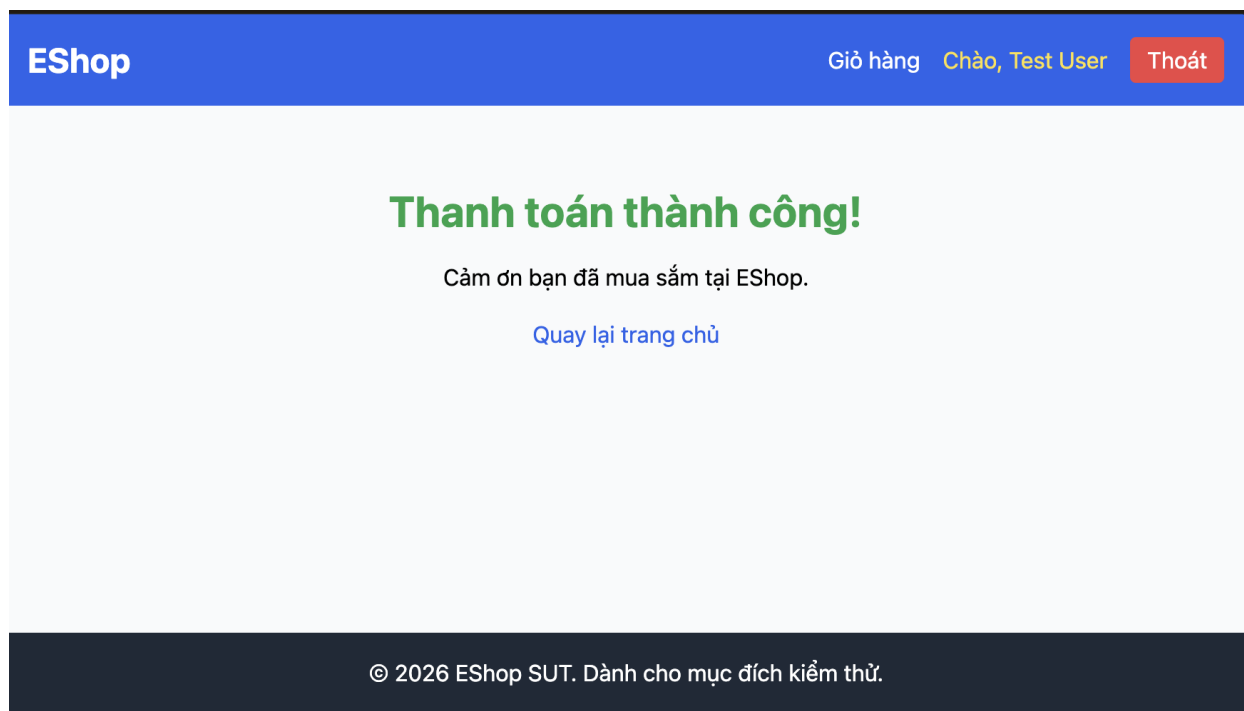
- User Guide
- Hướng dẫn DbUnit từng bước
- Hướng dẫn Database Rider từng bước
- Hướng dẫn kiểm thử Tonic.ai
- AI Audit Report

Bảng chứng screenshot seminar:

Bảng chứng DbUnit

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.371 s -- in com.eshop.dbunit.EshopDbUnitTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 0.953 s
[INFO] Finished at: 2026-07-03T22:44:51+07:00
[INFO] -----
jduckiess@192 dbunit-demo %
```

Bảng chứng DbUnit 1



Bảng chứng DbUnit 2



Bảng chứng DbUnit 3

```
jiduckiess@192 SeminarTesting % cd /Users/jiduckiess/Documents/SeminarTesting/dbunit-demo
mvn test
jiduckiess/.m2/repository/org/xerial/sqlite-jdbc/3.45.3.0/sqlite-jdbc-3.45.3.0.jar)
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

Total orders: 1
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.387 s -- in com.eshop.dbunit.EshopDbUnit
Test
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.224 s
[INFO] Finished at: 2026-07-04T00:17:01+07:00
```

Bảng chứng DbUnit 4

Bảng chứng Database Rider

```
jiduckiess@192 backend % cd /Users/jiduckiess/Documents/SeminarTesting/data
base-rider-demo
mvn test
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for call
ers in this module
WARNING: Restricted methods will be blocked in a future release unless nati
ve access is enabled

[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.60
6 s -- in com.eshop.databaserider.EshopDatabaseRiderTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.251 s
[INFO] Finished at: 2026-07-04T18:24:40+07:00
[INFO] -----

jiduckiess@192 database-rider-demo %
```

Bảng chứng Database Rider 1

```
jiduckiess@192 database-rider-demo % cd /Users/jiduckiess/Documents/Seminar
Testing
sqlite3 backend/database.sqlite "SELECT id, name, email, role FROM users;"
1|Admin User|admin@eshop.com|admin
2|Test User|test@eshop.com|user
jiduckiess@192 SeminarTesting %
```

Ln 280, Col 75

Bảng chứng Database Rider 2

Bảng chứng Tonic.ai

```
● jiduckiess@192 SeminarTesting % sqlite3 -header -csv backend/database.sqlite "SELECT * FROM users;" > tonic-export
t/source/users.csv
sqlite3 -header -csv backend/database.sqlite "SELECT * FROM products;" > tonic-export/source/products.csv
sqlite3 -header -csv backend/database.sqlite "SELECT * FROM coupons;" > tonic-export/source/coupons.csv
sqlite3 -header -csv backend/database.sqlite "SELECT * FROM orders;" > tonic-export/source/orders.csv
● jiduckiess@192 SeminarTesting % ls -la tonic-export/source
total 24
drwxr-xr-x@ 6 jiduckiess  staff  192 Jul  4 12:49 .
drwxr-xr-x@ 3 jiduckiess  staff   96 Jul  4 12:49 ..
-rw-r--r--@ 1 jiduckiess  staff  256 Jul  4 12:49 coupons.csv
-rw-r--r--@ 1 jiduckiess  staff    0 Jul  4 12:49 orders.csv
-rw-r--r--@ 1 jiduckiess  staff  700 Jul  4 12:49 products.csv
-rw-r--r--@ 1 jiduckiess  staff  194 Jul  4 12:49 users.csv
id,name,email,password,role,login_attempts,locked_until,reset_token,shipping_address,phone
1,"Admin User",admin@eshop.com,Admin123!,admin,0,,,,,
2,"Test User",test@eshop.com,Test1234!,user,0,,,,,
○ jiduckiess@192 SeminarTesting %
```

Bằng chứng Tonic.ai 1

```
tonic-export > source > users.csv
1 id,name,email,password,role,login_attempts,locked_until,reset_token,shipping_address,phone
2 1,"Admin User",admin@eshop.com,Admin123!,admin,0,,,,,
3 2,"Test User",test@eshop.com,Test1234!,user,0,,,,,
4
```

Bằng chứng Tonic.ai 2

The screenshot shows the Tonic.ai workspace interface. At the top, there's a navigation bar with options like 'Search workspaces', 'Workspaces', 'Add Data Source', 'Getting Started', 'Book A Demo', and '13 days left in free trial'. The workspace name 'Duc' is displayed, along with 'Owned by Đức Nguyễn Chí'. Below this, there's a sidebar with a 'New conversation' button and a chat area. The chat area shows a 'Hello!' message from the AI assistant. The main area displays an error message: 'Could not connect to source database'. The error message includes the text 'To connect a data source, edit the workspace.' and a red box with the message 'Connection Error: No Source DB has been configured for this Workspace'. There are buttons for 'Edit Workspace Settings' and 'View Docs'.

Bằng chứng Tonic.ai 3

Model

NAME

name » Name
Type : First Last

EMAIL

email » Email

PASSWORD

password » Constant
Value : MaskedPassword123!

SHIPPING_ADDRESS

shipping_address » Address
Type : Zip Code

PHONE

phone » Phone

Table: users

De-Identify

Preview

id	name	email	password	role	login_attempts	locked_until
1	Leif Krynsinski	sftyq@noted.tts	MaskedPassword123!	admin	0	
2	Ruthanne Gsell	bcma@qisfo.gnx	MaskedPassword123!	user	0	

Bảng chứng Tonic.ai 4

▶ Confirm Generation

×

Settings

Destination

Generation removes previous versions of the generated files, and creates new versions of the files based on the current generator settings.

☐

Enable Diagnostic Logging

?

☐

Send Logs to Tonic.ai

?

☐

Collect Performance Metrics

?

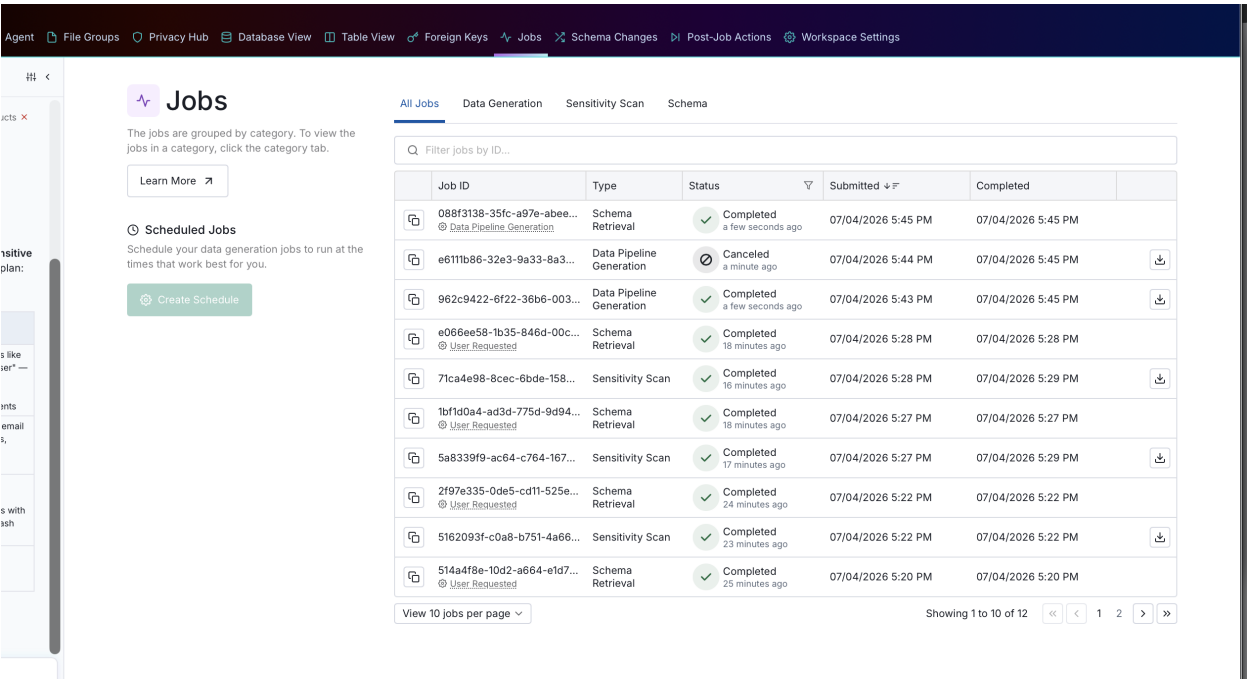
▶ Run Generation

▶▶ Want faster generations?

Use the suggested tips to make workspace and database changes that can improve generation performance.

Generation Tips

Bảng chứng Tonic.ai 5



Bằng chứng Tonic.ai 6

[← Back to All Jobs](#)

Data Pipeline Generation Job Details

Job ID: 962c9422-6f22-36b6-003d-5e8644eb64f3

Copy Job ID

Reports and Logs

Workspace

Untitled Workspace

Owner

ncduc23@clc.fitus.edu.vn

DB Type

Files

Subsetting

Not In Use

Post-Job Scripts

Not In Use

Webhooks

Not In Use

Worker Version

1795

Data available for file groups

De-identified files from this data generation job are available for download. Select a file group to download the files.

Download Results

Filter...

coupons

products

orders

users

Start Time

5:45:29 PM

Duration

0m 9s

Job Completed

Completed in a few seconds

Executing test suites

Completed in a few seconds

Showing slowest 2 tasks

Executing Source test suites completed in < 1 second

Executing Destination test suites completed in < 1 second

Total Time: < 1 second

Running pre-job checks

Completed in a few seconds

Applying generators to chained Fks

Completed in a few seconds

Copying passthrough table

Completed in a few seconds

Showing slowest 2 tasks

Copying File Group "coupons" completed in < 1 second

Bằng chứng Tonic.ai 7


```

2, test user, test@eshop.com, test123!, user, 0, , ,
● jiduckiess@192 SeminarTesting % ls -la tonic-export/generated
head -n 5 tonic-export/generated/users.csv
total 32
drwxr-xr-x@ 6 jiduckiess  staff  192 Jul  4 17:50 .
drwxr-xr-x@ 4 jiduckiess  staff  128 Jul  4 17:50 ..
-rw-r--r--@ 1 jiduckiess  staff  256 Jul  4 17:50 coupons.csv
-rw-r--r--@ 1 jiduckiess  staff  106 Jul  4 17:50 orders.csv
-rw-r--r--@ 1 jiduckiess  staff  688 Jul  4 17:50 products.csv
-rw-r--r--@ 1 jiduckiess  staff  228 Jul  4 17:50 users.csv
id,name,email,password,role,login_attempts,locked_until,reset_token,shipping_address,phone
1,Rebecca Cobler,mjwp@objcs.ndn,MaskedPassword123!,admin,0,,97708,
2,Leigha Batcher,xtyg@zxfgo.goy,MaskedPassword123!,user,0,,99547,
○ jiduckiess@192 SeminarTesting %

```

Bảng chứng Tonic.ai 8

```

tonic-export > generated > users.csv
1 id,name,email,password,role,login_attempts,locked_until,reset_token,shipping_address,phone
2 1,Rebecca Cobler,mjwp@objcs.ndn,MaskedPassword123!,admin,0,,97708,
3 2,Leigha Batcher,xtyg@zxfgo.goy,MaskedPassword123!,user,0,,99547,

```

Bảng chứng Tonic.ai 9

Các lệnh được sử dụng để tạo bảng chứng:

```

cd dbunit-demo
mvn test

```

```

cd database-rider-demo
mvn test

```

```

cd eshop-sut/backend
node database.js

```

15. Tài liệu tham khảo

- DbUnit: <https://www.dbunit.org/>
- Database Rider: <https://database-rider.github.io/database-rider/>
- Tonic.ai: <https://www.tonic.ai/>
- Tonic Structural data connectors: <https://docs.tonic.ai/app/setting-up-your-database/database-connectors>
- Tonic Structural MongoDB support: <https://docs.tonic.ai/app/setting-up-your-database/mongodb>
- Tổng quan Database Testing trên Viblo: <https://viblo.asia/p/database-testing-IA7GKnELMKZQ>